AA HANGAR · THE HANGAR

Agentic SDLC in Practice

A 3-Hour Hands-On Workshop
Constitutional AI for Enterprise Software Development

Adeel Ali · AI Coach & Advisor · The Hangar
AA Hangar AI Constitution

🛠 WORKSHOP · 3 HOURS · 65 SLIDES

4 Core Modules	**95min** Hands-On Practice
65 Slides	**3+** Live Exercises

Focus: Two exercises totaling 95 minutes of practice. Build with Constitutional AI governance — laws that AI agents MUST follow.

📋 Workshop Agenda

⏱️	**OPENING**	10 min
	└── Welcome, introductions, learning objectives	
📜	MODULE 1: Constitution Deep-Dive	35 min
	└── Laws, skills, workflows, authority hierarchy	
	└── Atomic TDD & VERIFY = 3 Gates	
🛠️	MODULE 2: Brownfield Adoption	45 min
	└── EXERCISE: Adopt Constitution to legacy service	
☕	**BREAK**	10 min
⚖️	MODULE 3: OpenSpec vs SpecKit	30 min
	└── DX comparison, token economics (50K vs 18K)	
🚀	MODULE 4: Agentic SDLC Step-by-Step	50 min
	└── EXERCISE: Build 2 vertical slices	
🎯	**CLOSING**	5 min
TOTAL DURATION		~3 hours

🎯 Learning Objectives

1️⃣ UNDERSTAND Constitutional AI

└── **The Hangar framework: Laws** ➔ Skills → Workflows

└── Authority hierarchy & NON-NEGOTIABLE laws

2️⃣ ADOPT the Constitution to Brownfield Code

└── 7-step adoption workflow

└── Generate constitutional artifacts with law citations

3️⃣ COMPARE OpenSpec vs SpecKit

└── Developer experience & token economics

└── When to use which (greenfield vs brownfield)

4️⃣ BUILD with Atomic TDD

└── **The 8-step cycle: RED** ➔ GREEN → REFACTOR → VERIFY

└── VERIFY = 3 Gates (Tests + Lint + Static Analysis)

Hands-On Focus: Two exercises totaling 95 minutes of practice!

🤝 How We'll Work Together

👤 HANGAR COACH

- Guides the narrative and big picture
- Facilitates discussions and Q&A
- Brings real-world experience

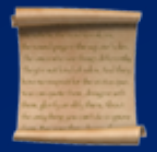
🤖 AI CO-FACILITATOR

- Explains concepts in depth on demand
- Guides exercises step-by-step
- Demonstrates live tooling and code

💬 INTERACTION COMMANDS

"Explain this slide" → Deep dive on current topic
"Demo this" → Live demonstration
"Start the exercise" → Step-by-step guidance

> *This is AI-human collaboration in action!*

 MODULE 1: Constitution Deep-Dive

Duration: 35 minutes

- **The 4 components: Laws** → Skills → Workflows → Adoptions

❓ The Problem

What Happens When AI Writes Code Without Guardrails?

- 🎭 **HALLUCINATIONS** AI confidently writes incorrect code
- 📊 **INCONSISTENT QUALITY** Great code one moment, terrible next
- 🔍 **NO AUDIT TRAIL** "Why did it do that?" — No one knows
- 🎯 **SCOPE CREEP** Builds more than asked
- 🔒 **SECURITY BLIND SPOTS** Forgets input validation

> *"AI is powerful, but power without governance is chaos."*

💡 The Solution: Constitutional AI

CONSTITUTIONAL AI

"Explicit rules that AI agents MUST follow"

📜 **LAWS**	→ What's allowed and forbidden
🛠️ **SKILLS**	→ HOW to do things correctly
🔄 **WORKFLOWS**	→ WHEN to apply which skills
🎯 **ADOPTIONS**	→ Customize for YOUR context

🏛️ Why "Constitutional"?

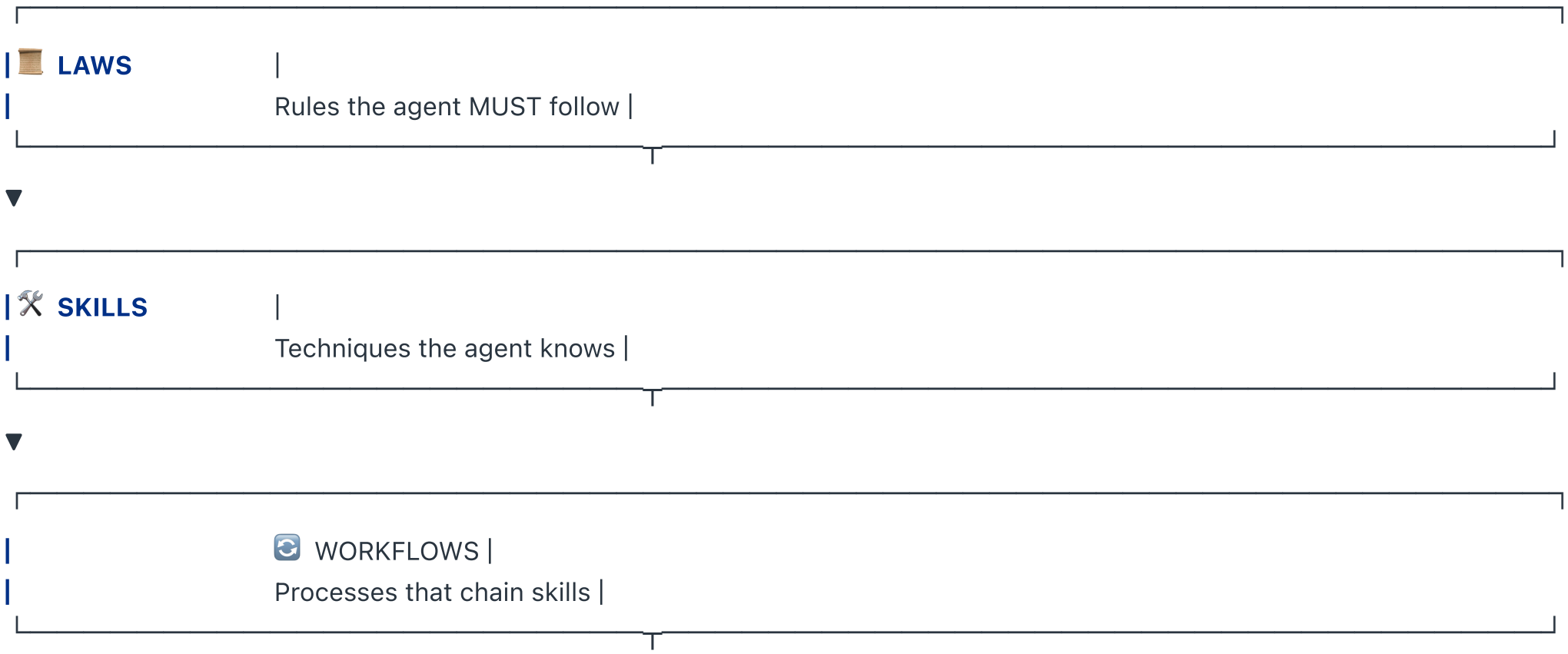
The Governance Analogy

GOVERNANCE CONCEPT	AI CONSTITUTION EQUIVALENT
Laws →	ENG-, *PRD*-, BUS-* laws
Amendments →	Law updates via committee review
Authority Hierarchy →	Laws → AGENTS.md → Project Rules
Enforcement →	constitution-lint, VERIFY gates

> *Just like a state constitution provides governance, our AI Constitution governs how agents behave — predictably and auditably.*

🏗️ The AA Hangar AI Constitution

Four Layers of Governance



⚖️ Authority Hierarchy

Who Wins When Rules Conflict?

LEVEL 1: CONSTITUTION LAWS (Highest Authority) 🔴

NON-NEGOTIABLE markers CANNOT be overridden

LEVEL 2: AGENTS.md (Operational) 🟠

Project-level customization within law bounds

LEVEL 3: Project Rules (Contextual) 🟡

Team conventions, style guides

LEVEL 4: OpenSpec Proposals (Work Context) 🟢

Current work in progress

> ***Key:*** *If a project rule conflicts with a constitutional law, the law ALWAYS wins.*

📂 Law Categories

Three Pillars of Governance

CATEGORY PREFIX	FOCUS EXAMPLES
Engineering ENG-*	How we build TDD, complexity
Product PRD-*	What we build User-first, OpenSpec
Business BUS-*	Why we build Compliance, audit trails

```
# Example from laws/index.yaml
- id: ENG-4.1
  title: Atomic TDD Law
  status: NON-NEGOTIABLE
```

🚫 NON-NEGOTIABLE Laws

These Cannot Be Overridden — Ever

LAW TITLE	WHY IT'S ABSOLUTE
ENG-4.1 Atomic TDD Law	"ONE test at a time" Prevents untested code
ENG-6.5	Input Validation Law Security cannot be optional
BUS-2.1	Aviation Compliance FAA/TSA/DOT are federal law

> ***NON-NEGOTIABLE*** *= No project rule, no deadline, no manager can override. The agent will REFUSE.*

🎭 Technology Avatars

Stack-Specific Guidance

AVATAR STACK	SPECIALIZES IN
java-spring	Java 21+, Spring 3.x Testing, DDD
python-fastapi	Python 3.11+, FastAPI Async, Pydantic
nodejs-typescript	TypeScript 5+, NestJS Type safety
react-frontend	React 18+, Next.js Component testing

Each avatar provides technology-specific law interpretations and code examples.

⚛️ Atomic TDD Law (ENG-4.1)

The Core Practice — NON-NEGOTIABLE

"ONE test at a time"

❌ WRONG	✅ RIGHT
Write 5 tests	Write 1 test
Then write code for all	Make it fail (RED)
Hope they all pass	Write minimum code (GREEN)
Refactor	
Repeat	

> ***Why?*** *One test at a time creates a tight feedback loop. You always know exactly what you're working on.*

🔄 The 8-Step Cycle

1. RED	→ Write ONE failing test
2. GREEN	→ Write MINIMUM code to pass
3. REFACTOR	→ Improve without changing behavior
4. VERIFY	→ Pass 3 Gates (Tests + Lint + Static)
5. UPDATE	→ Mark task complete in tasks.md
6. COMMIT	→ Conventional commit message
7. REPEAT	→ Next test in the slice
8. CELEBRATE	→ Acknowledge the increment! 🎉

> *Skipping steps breaks the cycle. The discipline IS the value.*

✅ VERIFY = 3 Gates

All Must Pass Before Commit

GATE 1: TESTS ✅

- All tests pass (green)
- Coverage meets threshold (≥90% for new code)

GATE 2: CONSTITUTION-LINT ✅

- npx constitution-lint check .
- No law violations

GATE 3: STATIC ANALYSIS ✅

- Cyclomatic complexity ≤ 10
- Cognitive complexity ≤ 7
- No critical issues

ALL 3 GATES MUST PASS BEFORE COMMIT

📊 Module 1 Recap

- ✅ CONSTITUTIONAL AI = Governance for AI-assisted development
- ✅ **FOUR COMPONENTS = Laws** ➔ Skills → Workflows → Adoptions
- ✅ AUTHORITY HIERARCHY = Laws always win over project rules
- ✅ NON-NEGOTIABLE = Cannot be overridden, agent will refuse
- ✅ ATOMIC TDD = ONE test at a time, always
- ✅ VERIFY = 3 GATES = Tests + Lint + Static Analysis

Questions before we move to the hands-on exercise?

🛠️ MODULE 2: Brownfield Adoption

Duration: 45 minutes (Exercise)

🏭 Why Brownfield First?

AA DEVELOPMENT REALITY

90% of our work is evolving EXISTING systems

We start where you ACTUALLY work

🔍 The Legacy Service

loyalty-service-legacy

```
// LoyaltyController.java — 270+ lines, 0% test coverage

@RestController
public class LoyaltyController {

    // ALL business logic in the controller
    // - Miles calculations
    // - Tier status determination
    // - Award redemption
    // ... everything in one file

    @PostMapping("/earn-miles")
    public ResponseEntity<?> earnMiles(...) {
        // 50+ lines of business logic
        // No validation, No error handling
        // Magic numbers everywhere
    }
}
```

Sound familiar? 🙋

What Makes It "Legacy"?

SMELL	IMPACT VIOLATION
0% test coverage	Can't refactor ENG-4.1, ENG-4.6 safely
All logic in controller	Untestable ENG-2.2, ENG-2.3
No input validation	Security vuln ENG-6.5
Magic numbers	Unclear rules ENG-3.1
No documentation	Knowledge silos ENG-6.1

📋 The 7-Step Adoption Workflow

1. ANALYZE	→ AI reads and understands the codebase
2. AGENTS.md	→ Create agent entry point with law citations
3. openspec/	→ Create specification folder structure
4. project.md	→ Document current state and context
5. proposal.md	→ Create change proposal with constitutional authority
6. tasks.md	→ Break into vertical slices with TDD cycles
7. ⛔ STOP	→ Review before ANY implementation

WHY STOP? The proposal is for REVIEW.

We don't write code without human approval.

🎯 The Trigger Prompt

Say This to Your AI Assistant

> I want to adopt the Hangar AI Constitution to this codebase.
>
> Please read the Brownfield Adoption Guide and begin
>
> the process.
>
> Use demo mode — pause at each step for teaching.

What Happens:

1. Agent reads the adoption guide
2. Analyzes the codebase
3. Pauses at each step for explanation
4. Generates artifacts with law citations
5. **Stops at proposal for review**

📁 Expected Outputs

```
loyalty-service-legacy/
│
├── AGENTS.md          → Agent entry point
│
└── openspec/
    │
    ├── project.md     → Current state documentation
    │
    └── changes/
        └── constitutional-adoption/
            │
            ├── proposal.md  → Why & what changes
            ├── design.md    → Architecture decisions
            ├── specs/       → BDD specifications
            └── tasks.md     → Vertical slices
```

AGENTS.md Structure

```
# AGENTS.md

## Constitutional Authority
This project operates under the AA Hangar AI Constitution.

## Project Context
- **Type:** Brownfield legacy service
- **Stack:** Java 11, Spring Boot 2.7
- **Current State:** 0% test coverage, monolithic controller

## Active Laws
| Law | Status | Adaptation |
|-----|--------|------------|
| ENG-4.1 | NON-NEGOTIABLE | Characterization tests first |
| ENG-2.3 | ACTIVE | Vertical slices for new features |

## Guardrails
- NEVER refactor without characterization tests
- ALWAYS cite constitutional authority for changes
```

📜 Constitutional Authority in Proposals

```
# Proposal: Constitutional Adoption for Loyalty Service

## Constitutional Authority

This proposal implements the following laws:

| Law ID     | Title                      | Relevance                      |
|------------|----------------------------|--------------------------------|
| ENG-4.1    | Atomic TDD Law             | Characterization tests first   |
| ENG-1.3    | Continuous Refactoring     | Strangler Fig for migration    |
| ENG-4.6    | Coverage Requirements      | Critical paths need 100%       |
| ENG-2.3    | Vertical Slice Law         | New features as slices         |

## Why This Change
The current codebase violates 5 constitutional laws...
```

> *Every proposal establishes which laws mandate this change.*

🚀 Exercise Time!

Brownfield Adoption (45 minutes)

SETUP

cd AA-Hangar-Constitution-Adoption-Test

cd loyalty-service-legacy

TRIGGER

"I want to adopt the Hangar AI Constitution to this codebase.

Please read the Brownfield Adoption Guide and begin

the process. Use demo mode — pause at each step."

GOAL

Generate all 7 artifacts with law citations

STOP at proposal for review

💬 Exercise Debrief

DISCUSSION QUESTIONS

1. What SURPRISED you about the adoption process?
2. Where did the agent CITE LAWS that you wouldn't have thought of?
3. Why do we STOP before implementation?
4. How does this compare to how you USUALLY approach legacy code?

☕ BREAK

10 Minutes

☕ ☕ ☕

Stretch, grab coffee, check messages

☕ ☕ ☕

COMING UP NEXT:

⚖️ Module 3: OpenSpec vs SpecKit

└── Token economics & why it matters

⚖️ MODULE 3: OpenSpec vs SpecKit

Duration: 30 minutes

🛠️ The Tool Landscape

SPEC-DRIVEN DEVELOPMENT TOOLS

KEY QUESTION: Which tool for which scenario?

📋 SpecKit: The Workflow

6 Rigid Commands in Sequence

/speckit.constitution → Define principles (MUST be first)
|
▼
/speckit.specify → Define requirements
|
▼
/speckit.plan → Select tech stack
|
▼
/speckit.tasks → Generate task list
|
▼
/speckit.implement → Build from scratch
|
▼

😬 SpecKit: Developer Experience

SCENARIO	EXPERIENCE
Start a new project	✅ Great! Linear flow
Change requirements mid-work	😬 Restart the cycle
Add feature to existing system	😬 Must specify entire system context
Iterate on design	😬 Regenerate all artifacts
Review audit trail	❓ Implicit in history

Best For: True 0→1 greenfield projects where requirements are stable.

📋 OpenSpec: The Workflow

9 Flexible Commands in Any Order

/opsx:explore [topic] → Investigate before committing
/opsx:new [name] → Create new change proposal
/opsx:continue [name] → Generate next artifact
/opsx:ff [name] → Fast-forward: generate all at once
/opsx:apply [name] → Execute implementation tasks
/opsx:verify [name] → Validate implementation
/opsx:sync [name] → Integrate delta specs
/opsx:archive [name] → Complete and archive

✅ Use ANY order — match how you actually think and work

✅ OpenSpec: Developer Experience

SCENARIO	EXPERIENCE
Start a new feature	✅ Create proposal, iterate freely
Change requirements mid-work	✅ Update proposal, continue
Add to existing system	✅ Delta spec (changes only)
Iterate on design	✅ Incremental updates
Review audit trail	✅ Explicit in changes/

Best For: Enterprise brownfield work (90% of AA development).

🔄 DX Comparison Side-by-Side

TASK	SPECKIT OPENSPEC
Start a feature sequence	6-command Create proposal freely
Change requirements	Restart cycle Update & continue
Add to existing system	Specify entire Delta spec only
Iterate on design	Regenerate all Incremental
Audit trail	Conversation changes/ folder
Parallel features	One at a time As many as needed

💰 Why Tokens Matter

TOKENS = THE CURRENCY OF AI

┌──┐
| |
| **CONTEXT WINDOW LIMITS** |
| ── |
| Every AI has a maximum context (128K, 200K tokens) |
| **Fill it up** → AI forgets earlier context |
| **Response quality DEGRADES** |
| |
| **API COSTS** |
| ──────────────── |
| **Tokens = money** |
| **More tokens = higher costs** |
| Regenerating = paying AGAIN for same information |
| |
└──┘

"Tokens = money = sustainability"

🎯 Live Demo: Token Optimization

Practice Guide Prompt

Copy and run this prompt to see token optimization in action:

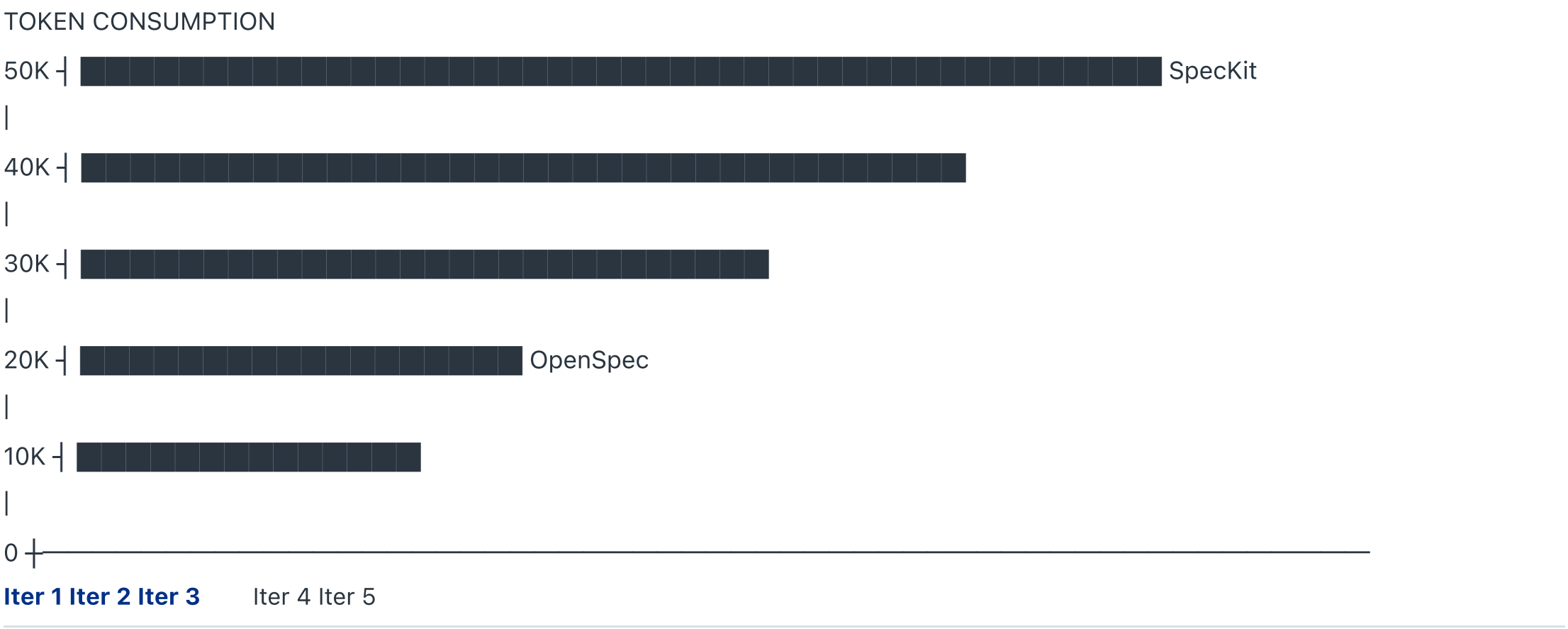
> I'm working on a Java Spring Boot project and need help with:
>
> 1. Writing unit tests for a service
> 2. Following proper test naming conventions
> 3. Using the right Spring test annotations
>
> Based on the AA-Hangar-AI-Constitution token-optimized structure, which files should you load?
>
> Explain why each file is needed.

Expected AI Behavior:

1. Loads `avatars/index.yaml` first (router)
2. Identifies `java-spring` as relevant avatar
3. Loads only `avatars/technology/java-spring.md`
4. Loads only relevant laws (ENG-4.1, ENG-4.2)

The Token Graph

SpecKit vs OpenSpec Over 5 Iterations



🔄 Regenerative vs Delta

SPECKIT: REGENERATIVE MODEL

Each iteration regenerates ALL artifacts

Token cost: O(n) — linear growth

5 iterations × 10K tokens = 50K tokens

OPENSPEC: DELTA MODEL

Each iteration sends ONLY changes

Token cost: O(1) per change — constant

Initial 10K + (4 iterations × 2K) = 18K tokens

64% SAVINGS × hundreds of engineers × thousands of iterations

= SIGNIFICANT cost and quality impact

⚠ The Waterfall Parallel

WINSTON ROYCE (1970)

Presented waterfall but noted it had "major flaws"

and was "risky and inviting failure" for iteration.

THE PATTERN:

Stage	Waterfall	SpecKit Risk
Design Intent	Initial flow	Greenfield only
Adoption Scope	Applied to ALL	Risk of misuse
Consequence	Decades of fail	Token waste

DOD-STD-2167 (1985): Mandated rigid waterfall → decades fail

MIL-STD-498 (1994): Reversed course, encouraged iteration

🎯 Token Optimization: Multi-RAG

FULL CONSTITUTION: ~549,250 TOKENS

(Exceeds ALL AI context limits!)

WITH SELECTIVE LOADING: ~12,700 TOKENS

(97.7% reduction)

3-LEVEL RETRIEVAL STRATEGY:

Level 1: Catalog lookup (index files) ~6K tokens

Level 2: Selective skill/workflow loading ~4-8K per skill

Level 3: Avatar specialization ~2-5K per avatar

Key: "Load selectively, not comprehensively"

✅ Module 3 Recap: Decision Tree

Is this greenfield (0→1)?

|

├── **YES** → SpecKit or Copilot Workspace may be fine

|

└── **NO (brownfield, 1→n)** → OpenSpec is the right tool

KEY TAKEAWAYS:

- ✅ OpenSpec: Delta-based, 64% token savings, brownfield-native
- ✅ SpecKit: Regenerative, good for true greenfield
- ✅ **AA is 90% brownfield** → OpenSpec is our default
- ✅ Token consumption = cost + quality + sustainability

🚀 MODULE 4: Agentic SDLC Step-by-Step

Duration: 50 minutes (Exercise)

🔄 The 5-Phase Flow

PHASE 1: ADOPT THE CONSTITUTION

Clone repo, create AGENTS.md, run linter

|

▼

PHASE 2: SELECT DOMAIN & TECH STACK

Product avatar (Cargo, Loyalty) + Tech avatar (Python, Java)

|

▼

PHASE 3: CHOOSE A SPECIFICATION

AI generates 3 options + custom option

|

▼

PHASE 4: GENERATE CONSTITUTIONAL PROPOSAL

proposal.md, design.md, specs/, tasks.md

|

🎭 Phase 2: Select Avatars

PRODUCT AVATARS (Domain)

- **Cargo** — Freight booking, tracking, capacity
- **Loyalty** — AAdvantage miles, tier status, awards
- Flight Ops — Scheduling, crew, maintenance

TECHNOLOGY AVATARS (Stack)

- Python/FastAPI — Async, Pydantic, pytest
- **Java/Spring** — Spring Boot, JUnit, DDD patterns
- TypeScript/Node — NestJS, Jest, decorators

> *Choose based on what's most relevant to YOUR work.*

📋 Phase 3: Choose a Specification

Based on your selected avatars, here are 3 spec options:

1. WEATHER DASHBOARD

Real-time weather data with caching and alerts

Slices: Fetch → Cache → Display → Alert

2. CARGO TRACKING API

Shipment status with event sourcing

Slices: Create → Track → Notify → Report

3. MILES CALCULATOR

Earn/redeem with tier multipliers

Slices: Earn → Redeem → Tier → History

4. CUSTOM

Describe your own feature

📁 Phase 4: Constitutional Proposal

```
openspec/
 └── changes/
 └── weather-dashboard/ # (or your chosen spec)
|
├── proposal.md # Constitutional authority
|
├── design.md # Architecture decisions
|
├── specs/
| ├── fetch-weather.feature
| ├── cache-data.feature
| └── display-dashboard.feature
|
└── tasks.md # Implementation roadmap
```

📋 Understanding tasks.md

```
# Tasks — Weather Dashboard

## Status: Ready for Implementation

*Per ENG-1.4 (Vertical Slice) and ENG-4.1 (Atomic TDD)*
```

Slice 1: Fetch Weather Data

- [ ] 1.1 **RED** — Write failing test for WeatherService.fetch()
- [ ] 1.2 **GREEN** — Implement minimum fetch logic
- [ ] 1.3 **REFACTOR** — Extract WeatherData value object
- [ ] 1.4 **VERIFY** — All 3 gates pass
- [ ] 1.5 **COMMIT** — `feat(weather): add fetch capability`

MVP BOUNDARY

Slice 1 is a complete vertical slice. Stop here if time is limited.

🔴 Phase 5: RED

Write ONE Failing Test

```
# tests/test_weather_service.py

def test_fetch_returns_weather_data_for_valid_city():
    """
    Given a valid city name
    When fetching weather data
    Then it returns temperature and conditions
    """
    service = WeatherService()

    result = service.fetch("Dallas")

    assert result.temperature is not None
    assert result.conditions in ["sunny", "cloudy", "rainy"]
```

```
pytest tests/test_weather_service.py -v
# Expected: FAILED (WeatherService doesn't exist yet)
```

> *If it passes, you're doing TAD (Test After Development), not TDD!*

🟢 Phase 5: GREEN

Write MINIMUM Code to Pass

```
# app/weather_service.py

from dataclasses import dataclass

@dataclass
class WeatherData:
    temperature: float
    conditions: str

class WeatherService:
    def fetch(self, city: str) -> WeatherData:
        # MINIMUM implementation - hardcoded for now
        return WeatherData(temperature=75.0, conditions="sunny")
```

```
pytest tests/test_weather_service.py -v
# Expected: PASSED
```

> *Resist the urge to optimize. Refinement comes in REFACTOR.*

♻️ Phase 5: REFACTOR

Improve Without Changing Behavior

```
# app/weather_service.py

from dataclasses import dataclass
from typing import Literal

Conditions = Literal["sunny", "cloudy", "rainy", "snowy"]

@dataclass(frozen=True)  # Immutable per ENG-3.2
class WeatherData:
    """Value object representing weather conditions."""
    temperature: float
    conditions: Conditions
    city: str

class WeatherService:
    """Service for fetching weather data."""

    def fetch(self, city: str) -> WeatherData:
        return WeatherData(temperature=75.0, conditions="sunny", city=city)
```

✅ Phase 5: VERIFY

All 3 Gates Must Pass

```
# Gate 1: Tests
pytest --cov=app --cov-report=term-missing
# Expected: All tests pass, coverage ≥ 90%

# Gate 2: Constitution-lint
npx constitution-lint check .
# Expected: No violations

# Gate 3: Static analysis
ruff check . && mypy app/
# Expected: No critical issues
```

✅ Gate 1: Tests PASSED (coverage 94%)

✅ Gate 2: Constitution-lint PASSED (0 violations)

✅ Gate 3: Static analysis PASSED (complexity 3)

ALL GATES PASSED — Ready to commit!

📝 Phase 5: UPDATE & COMMIT

Update tasks.md:

```
## Slice 1: Fetch Weather Data

- [x] 1.1 **RED** — Write failing test for WeatherService.fetch()
- [x] 1.2 **GREEN** — Implement minimum fetch logic
- [x] 1.3 **REFACTOR** — Extract WeatherData value object
- [x] 1.4 **VERIFY** — All 3 gates pass
- [x] 1.5 **COMMIT** — `feat(weather): add fetch capability`
```

Commit:

```
git add -A
git commit -m "feat(weather): add fetch capability

- WeatherService.fetch() returns WeatherData
- Value object immutable per ENG-3.2
- Coverage: 94%

Refs: ENG-4.1, ENG-3.2"
```

🎉 Slice 1 Complete!

✅ SLICE 1: FETCH WEATHER DATA

- Test written and passing
- Minimum implementation complete
- Code refactored with value objects
- All 3 VERIFY gates passed
- Tasks updated
- Committed with constitutional references

This is a COMPLETE vertical slice — shippable increment!

🔄 Continue to Slice 2

```
## Slice 2: Cache Weather Data

- [ ] 2.1 **RED** — Write failing test for caching behavior
- [ ] 2.2 **GREEN** — Implement simple cache
- [ ] 2.3 **REFACTOR** — Extract CacheService
- [ ] 2.4 **VERIFY** — All 3 gates pass
- [ ] 2.5 **COMMIT** — `feat(weather): add caching`
```

The cycle repeats:

```
RED → GREEN → REFACTOR → VERIFY → UPDATE → COMMIT → REPEAT
```

> *Same discipline, new functionality. The rhythm becomes natural.*

🚀 Exercise Time!

Agentic SDLC Step-by-Step (50 minutes)

SETUP

cd AA-Hangar-Agentic-SDLC-Workshop

TRIGGER

"Help me run the AA Hangar Agentic SDLC Workshop from WORKSHOP-GUIDE.md"

GOAL

Complete all 5 phases

Implement at least 2 vertical slices

Pass all 3 VERIFY gates for each slice

📊 MVP Boundary

COMPLETED FUTURE SLICES

✅ **Slice 1: Fetch Weather** ☐ Slice 3: Display Dashboard

✅ **Slice 2: Cache Data** ☐ Slice 4: Weather Alerts

☐ Slice 5: Historical Data

—— MVP BOUNDARY ——————————————————————————————

KEY INSIGHT:

We have 2 COMPLETE vertical slices

Each is shippable, tested, documented

Better than 5 half-finished features!

💬 Exercise Debrief

DISCUSSION QUESTIONS

1. How did the 8-step cycle FEEL?

Natural or awkward?

2. Where did VERIFY catch issues you might have missed?

3. How does TASK TRACKING change your awareness of progress?

4. What would you do DIFFERENTLY in your real projects?

🎯 CLOSING

Duration: 5 minutes

What we'll cover:

- The 4 metrics that matter
- Anti-metrics to avoid
- Your next steps
- Resources and community

📊 The 4 Metrics That Matter

METRIC	MEASURES TARGET
Defect Escape Rate	Bugs reaching production < 5%
Time to Productivity	New dev to first commit < 2 weeks
Agentic SDLC Compliance	Constitutional adherence > 90%
"AI Off" Competency	Skills without AI Maintained

> *These track **quality and sustainability**, not just speed.*

⚠️ Anti-Metrics to Avoid

ANTI-METRIC	WHY IT'S HARMFUL
Lines of code	Incentivizes bloat
Story points velocity	Goodhart's Law — becomes target
AI usage rate	Quantity over quality
Time to first commit	Skips verification
Features per sprint	Ignores completeness

> *"When a measure becomes a target, it ceases to be good." — Goodhart*

📚 Take-Home Resources

PRACTICE GUIDES (~30 min each)

- aa-engineering-laws/practice-guides/atomic-tdd/
- aa-engineering-laws/practice-guides/vertical-slice/
- aa-engineering-laws/practice-guides/token-optimization/

TECHNOLOGY AVATARS

- aa-engineering-laws/adoptions/java-spring/
- aa-engineering-laws/adoptions/python-fastapi/
- aa-engineering-laws/adoptions/react-frontend/

COMMUNITY

- AI Community of Practice Slack channel
- Monthly Constitution Office Hours

🚀 Your Next Steps

WEEK 1

- ☐ Adopt Constitution to ONE real project
- ☐ Complete Atomic TDD practice guide (30 min)
- ☐ Run constitution-lint on your codebase

WEEK 2

- ☐ Implement ONE vertical slice with full cycle
- ☐ Share experience in AI CoP channel
- ☐ Review your team's AGENTS.md

MONTH 1

- ☐ Track Defect Escape Rate before/after
- ☐ Mentor a colleague through adoption
- ☐ Propose one law improvement

? Q&A

COMMON QUESTIONS:

- "How do I get my team onboarded?"
- "What if my manager wants to skip tests?"
- "How do I handle deadline pressure?"
- "What about legacy systems with no tests?"
- "How do I contribute to the Constitution?"

What questions do you have?

🙏 Thank You!

WHAT YOU LEARNED TODAY

- ✅ Constitutional AI & the Hangar framework
- ✅ Brownfield adoption workflow
- ✅ OpenSpec vs SpecKit & token economics
- ✅ Atomic TDD & VERIFY = 3 Gates

RESOURCES

- AA-Hangar-AI-Constitution repository
- AI Community of Practice channel
- constitution-lint: npm install -g @anthropic/constitution-lint

CONTACT: ai-cop@aa.com

Go forth and build with Constitutional AI! 🚀